

“Al infinito...Y más allá”

Video juego inspirado en la película ToyStory y el atletismo.

Valentina Posada Vásquez

Natalia Zapata Zapata

Introducción

“Al infinito...y más allá” es un videojuego el cual creamos tomando como inspiración el universo de Toy Story y el deporte que decidimos representar fue el atletismo. En este videojuego el usuario controla a Woody el cual es nuestro personaje principal para correr, saltar, recoger premios y evitar obstáculos o piedras que son lanzadas por un enemigo inteligente. Este desarrollo fue llevado a cabo para demostrar programación orientada a objetos, uso de memoria dinámica, interfaz gráfica, herencia propia, manejo de excepciones y aplicación de diferentes modelos físicos dentro de una mecánica jugable. La lógica del juego integra tres modelos físicos principales los cuales no dependen del movimiento rectilíneo uniforme: el movimiento parabólico para saltos y lanzamiento de piedras, la ley de coulomb para atraer premios cuando se toma el juguete premiado y la segunda ley de Newton para calcular el daño asociado a las piedras ya que depende de el tamaño de las mismas es proporcional al daño ocasionado. Además, el agente inteligente se diseñó con una lógica de estados que le permite percibir la posición del jugador, mantener la distancia, acercarse o lanzar piedras de acuerdo con la dificultad configurada.

El videojuego no tiene público objetivo debido a que el sistema de dificultad es configurable lo que lo hace accesible a distintos perfiles y respecto al ámbito académico el proyecto está dirigido o enfocado a evidenciar competencias y usos adecuados en C++, Qt y programación orientada a objetos donde se abarque todo lo visto en el curso.

Planteamiento del problema

El desarrollo de este videojuego representó un gran reto académico porque requiere la integración de la programación orientada a objetos, interfaces gráficas, físicas aplicadas y uno de los conceptos más importantes del curso que es la gestión de recursos dentro de la aplicación funcional. Uno de los principales desafíos fue la

integración de conceptos un poco más avanzados en C++, incluyendo herencia, memoria dinámica, manejo de excepciones e interacciones físicas dentro de un entorno que fuese atractivo y comprensible para cualquier tipo de usuario.

Definición general y objetivos

Los objetivos generales que nos propusimos para abarcar la totalidad los conceptos esenciales fueron: Diseñar una arquitectura modular la cual fue basada en clases, implementar una jerarquía propia de herencia, utilizar adecuadamente la memoria dinámica, incorporar sprites, sonidos, menú principal e incluso créditos para una correcta interacción, la aplicación de los modelos físicos movimiento parabólico, Ley de Coulomb y la segunda ley de Newton. Nuestro alcance respecto a lo funcional consistió en controlar a Woody mediante el teclado, seleccionar la dificultad de los niveles, activar poderes eléctricos temporales, evitar y evadir obstáculos tanto estáticos como dinámicos, etc.

Especificación de requerimientos

En el aspecto funcional, el sistema permite iniciar nuevas partidas, seleccionar niveles de dificultad, control total del personaje principal, gestionar colisiones, el uso de timers para la aparición de obstáculos y premios y finalizar la partida cuando se cumplan las condiciones establecidas. Además, se proporcionan mecanismos para actualizar la puntuación, controlar el nivel de energía y administrar el comportamiento del enemigo inteligente.

Metodología y planificación

El desarrollo y planificación lo realizamos basándonos en la reconstrucción progresiva de las diferentes funcionalidades. Inicialmente llevamos a cabo la etapa del análisis de requisitos con el fin de identificar los requisitos para cumplir los objetivos, esto se realizó en el Momento I. Luego de establecer nuestras ideas dimos paso a la implementación de las entidades, la interfaz gráfica y los sistemas de control del juego. Luego de esto fuimos incorporando los modelos físicos, la inteligencia que debía tener el enemigo para finalmente realizar pruebas y ajustes del correcto funcionamiento.

Diseño y arquitectura del sistema

La arquitectura la diseñamos siguiendo un enfoque modular en el cual se separan claramente las responsabilidades de cada uno de los componentes. La clase MainWindow es la que elegimos para que actuara como la clase principal, esta administra diferentes funciones como lo es el menú principal, la selección de dificultad. Por otra parte, GameScene toma el papel de núcleo porque actualiza las entidades, procesa la interacción con el usuario y controla las colisiones. A partir de

esta estructura se derivan las demás clases como lo son el jugador, el enemigo, los obstáculos, las físicas, etc.

Con este diseño de arquitectura podemos mantener una separación entre la lógica, la representación gráfica y la administración de recursos lo cual favorece la reutilización de código y facilita futuras ampliaciones del proyecto. Adicionalmente se emplean componentes de la biblioteca estándar de C++, incluyendo contenedores dinámicos como `std::vector` y mecanismos modernos de administración de memoria como `std::unique_ptr`. La compilación y configuración del proyecto se gestionan mediante CMake, lo que facilita la portabilidad y organización del sistema.

Desarrollo e implementación

La implementación del videojuego se fundamenta en una arquitectura orientada a objetos donde cada entidad posee responsabilidades claramente definidas. El funcionamiento general es dependiente de un bucle que se ejecuta periódicamente por medio de timers. Durante cada iteración se procesan las entradas del usuario, se actualizan las posiciones de las entidades, se aplican las físicas correspondientes a las acciones, se verifican colisiones y procedemos a actualizar los elementos visuales de la interfaz. Algo relevante de nuestro proyecto es que hicimos uso de punteros inteligentes los cuales evitan fugas de memoria, se encargan de los cálculos físicos en una clase específica y se controla el comportamiento del enemigo. Todo lo anterior hace parte de la modularización y un diseño más limpio.

Procedimientos de prueba

Las pruebas que realizamos tuvieron como objetivo verificar tanto el funcionamiento individual de los componentes como la integración completa del juego. Pusimos a prueba el correcto funcionamiento del menú principal y los comportamientos de los personajes. Asimismo, realizamos pruebas relacionadas con la estabilidad de la compilación debido a que hacemos uso de diferentes recursos como lo son la parte gráfica y la parte auditiva. Gracias a la realización de estas pruebas pudimos identificar y corregir problemas relacionados con la velocidad de las piedras lanzadas por el enemigo, la distancia entre los obstáculos y la respuesta a los movimientos de Woody logrando así una experiencia en el juego estable.

Guía de instalación y uso

Durante la partida, el usuario puede desplazarse utilizando las teclas de dirección o las teclas A y D, realizar saltos mediante la barra espaciadora o la flecha superior, poner pausa al juego con la tecla Esc y regresar al menú presionando la tecla M. El objetivo consiste en sobrevivir el mayor tiempo posible mientras se recogen premios y se eviten obstáculos y piedras que lanza el enemigo.

Resultados y discusión

Como resultado del proceso de desarrollo obtuvimos un videojuego funcional capaz de integrar múltiples conceptos de programación y todo dentro de una misma aplicación. Se logró implementar una interfaz gráfica interactiva, modelos físicos aplicados a la jugabilidad, un agente inteligente el cual se basa en estados y un sistema configurable respecto a su dificultad.

Como se expresó anteriormente, durante el desarrollo surgieron diferentes dificultades las cuales fueron abordadas mediante procesos de pruebas y ajustes. No obstante, aún existen posibilidades de ampliación relacionada con nuevos niveles, mecánicas adicionales, mejoras visuales y algoritmos un poco más avanzados para la inteligencia del enemigo.

Conclusiones

El desarrollo de este proyecto permitió consolidar conocimientos fundamentales de programación orientada a objetos, diseño de software, administración de memoria y desarrollo de interfaces gráficas utilizando herramientas un poco más avanzadas. La implementación de modelos físicos e inteligentes nos mostró cómo los conceptos teóricos pueden integrarse de manera efectiva dentro de una aplicación interactiva.

Además de cumplir los objetivos académicos que nos fueron planteados, el proyecto permitió fortalecer habilidades relacionadas con el análisis de problemas, el diseño de arquitecturas modulares, la depuración de errores e incluso una sintaxis técnica. Nos dejó como enseñanza que para futuros desarrollos puede resultar conveniente continuar aspectos muy importantes como lo son la optimización aprovechando las bases que pudimos solidificar con la ejecución de este videojuego.

Referencias

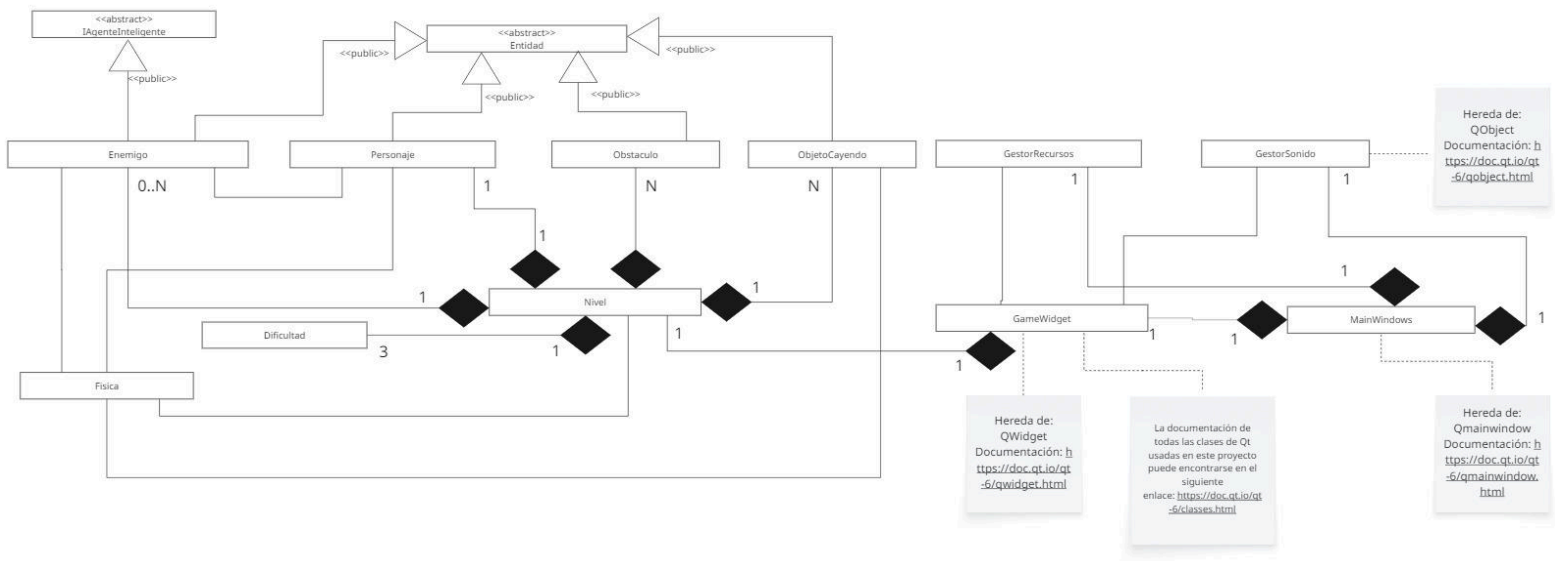
1. Cppreference. (s.f.). *Cppreference.com*. Recuperado el 6 de junio de 2026, de [cppreference.com](https://en.cppreference.com)
2. The Spriters Resource. (s.f.). *Toy Story (SNES) sprites*. Recuperado el 6 de junio de 2026, de [The Spriters Resource - Toy Story \(SNES\)](https://www.spriters-resource.com/snes/toystory/)

Anexos/Apéndices

Link directo para mejor visualización: <https://miro.com/app/board/uXjVHZvgxt0/>



Diagrama de clases resumen



Link directo para mejor visualización: https://miro.com/app/board/uXjVHc4Sawg=